

Constraint Optimization through RTL Enhancement Using Generative AI

Abstract

Timing closure is one of the most persistent bottlenecks in modern digital IC design. As designs grow to incorporate deep pipelines, wide datapaths, and tightly coupled control logic, meeting timing constraints after synthesis increasingly requires revisiting the Register-Transfer Level (RTL) itself, rather than relying on logic synthesis alone. Today, this RTL-level correction is performed almost entirely by hand: engineers inspect post-synthesis timing reports, manually trace critical paths back to specific lines of RTL, apply restructuring techniques such as pipelining, retiming, or logic re-balancing, and re-run synthesis to verify improvement – repeating this cycle until timing targets are met. This process is slow, does not scale with design complexity, depends heavily on individual engineer expertise, and explores only a tiny fraction of the vast space of functionally equivalent RTL rewrites, often leaving significant performance, power, and area (PPA) improvement on the table.

This work addresses the problem by using Generative AI to automate RTL-level timing optimization. We survey and adopt the design principles proposed in *Dr. RTL: Autonomous Agentic RTL Optimization through Tool-Grounded Self-Improvement* (Fang et al., HKUST, 2026), which formulates RTL optimization as a closed-loop, tool-grounded agentic search problem rather than a single-shot code generation task. In this framework, an orchestrator coordinates three specialized agents – a *Timing Analysis Agent* that localizes critical-path bottlenecks from fine-grained post-synthesis timing reports, an *RTL Optimization Agent* that proposes multiple functionally equivalent RTL rewrites in parallel, and an *Evaluation Agent* that verifies each candidate through re-synthesis and formal equivalence checking. Crucially, the framework does not merely optimize a single design in isolation: through *group-relative skill learning*, it compares parallel candidates generated under identical conditions, distills the transformations that consistently outperform their peers into a reusable, human-interpretable skill library of recurring timing-bottleneck patterns and effective fix strategies, and reuses this accumulated knowledge across future designs and iterations. This combination of fine-grained EDA feedback, parallel exploration, and continual skill accumulation allows the system to substantially outperform both single-shot LLM prompting and prior rule-based RTL optimization methods, while formal equivalence checking (via the open-source EQY/SymbiYosys toolchain in our adaptation) guarantees that every accepted rewrite is functionally identical to the original design. Our proposed solution adapts this agentic, self-improving methodology to an open-source EDA toolchain (Yosys, OpenSTA, OpenROAD, EQY), targeting realistic multi-clock RTL designs, with the goal of demonstrably reducing timing violations and manual engineering effort while preserving correctness.

1 Understanding the Problem

Modern System-on-Chip (SoC) and IP designs are described in RTL using hardware description languages such as Verilog or SystemVerilog. RTL captures a design’s intended behavior at the register/clock-cycle level, but it is not yet a physical circuit – it must pass through a well-defined **synthesis and physical design flow** before it can be fabricated. A central requirement at every stage of this flow is **timing closure**: every signal path in the design must complete its propagation, from one register to the next, within a single clock period, across all process, voltage, and temperature (PVT) corners and across every clock domain in the design (including designs with multiple independent, asynchronous clocks). When a path fails to do so, it is reported as a **timing violation**, and the design cannot be safely operated at its target frequency.

Why This Matters

Timing violations are not cosmetic issues – an unresolved violation can cause a chip to malfunction unpredictably in real silicon (metastability, incorrect data capture, functional failures under certain operating conditions). At the same time, being overly conservative to *guarantee* timing closure wastes area, power, and performance headroom. Achieving the right balance – meeting timing with minimal area/power overhead – is what makes timing closure simultaneously critical and difficult.

2 The RTL-to-Layout Flow and the Role of Timing Reports

Figure 1 illustrates the standard flow that a design goes through, and where timing feedback re-enters the process.

1. **RTL Design.** Engineers write Verilog/SystemVerilog describing the intended hardware behavior.
2. **Logic Synthesis.** A synthesis tool (e.g., Yosys, or a commercial tool such as Synopsys Design Compiler) translates RTL into a gate-level netlist mapped to a standard-cell library.
3. **Static Timing Analysis (STA).** A timing tool (e.g., OpenSTA) reads the netlist together with a Library (.lib) file (cell timing models) and an SDC file (clock and I/O timing constraints), and computes the delay of every path in the design.
4. **Timing Report.** STA produces a report listing, for every critical path, its **slack** (positive = timing met, negative = violation), the **Worst Negative Slack (WNS)** and **Total Negative Slack (TNS)** across the design, and the exact chain of gates/registers that make up the offending path. This is the artifact that both human engineers and the proposed GenAI system consume to decide what to fix.
5. **Decision Point.** If violations remain, the design must be revised at the RTL level (not just re-synthesized) and the loop above repeats. If timing is met, the design proceeds to physical design.
6. **Physical Design (Place & Route).** Tools such as OpenROAD perform floorplanning, placement, clock-tree synthesis, and routing, ultimately producing a GDSII layout ready for fabrication. Post-route timing is re-checked, since real wire delays can reintroduce violations not visible at the synthesis stage.

3 The Long, Manual Way – and Its Problems

Step 5 above – *manually revising RTL in response to a timing report* – is the crux of the problem. In current industrial practice, this step looks like:

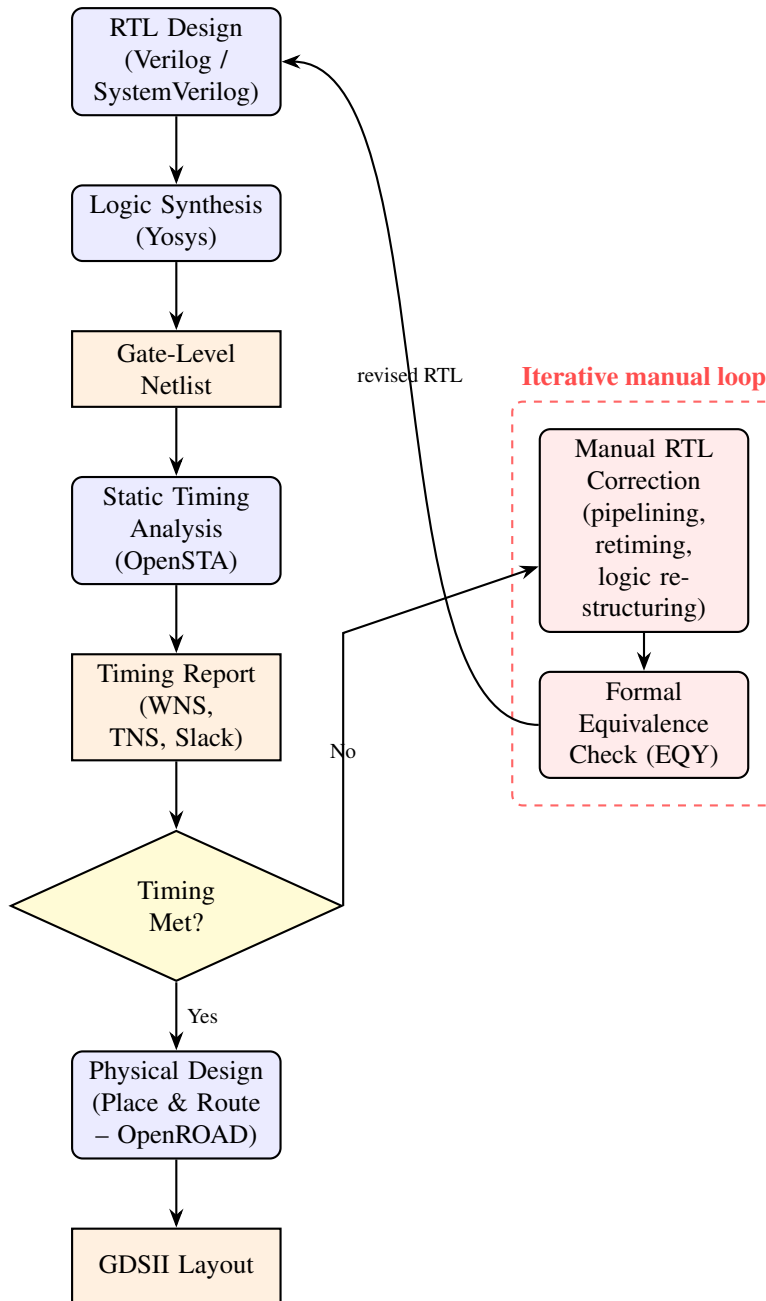


Figure 1: The RTL-to-Layout design flow, highlighting the iterative timing-driven loop between synthesis, static timing analysis, and manual RTL correction that this project targets for automation.

1. An engineer reads a timing report and manually maps the reported gate-level critical path back to the RTL source lines that produced it (a non-trivial task once technology mapping and optimization have transformed the logic).
2. The engineer reasons about *why* the path is slow – e.g., excessive combinational depth, high fan-out, a wide comparison, deep FSM/decode logic, or reconvergent logic – and selects a fix, such as inserting a pipeline register, retiming, pre-computing a condition, or restructuring a mux tree.
3. The RTL is edited by hand, and the *entire* synthesis → STA loop is re-run to check whether the fix actually helped – often only to find that the fix was absorbed by synthesis, moved the bottleneck elsewhere, or (worse) broke functional correctness.
4. Functional correctness must then be re-verified, ideally through **formal equivalence checking** (prov-

ing the new RTL is logically identical to the original for all inputs) rather than simulation alone, since simulation can only test a finite set of input vectors and can miss corner-case bugs introduced by the rewrite – this is especially important for asynchronous clock-domain-crossing logic, where bugs are notoriously difficult to catch by simulation.

This creates several compounding problems: the process is **labor-intensive and slow**, since each iteration requires a full synthesis and timing run; it is **expertise-bottlenecked**, since good fixes rely on tacit knowledge that experienced engineers accumulate over years and rarely document systematically; it **explores an infinitesimally small fraction** of the space of functionally equivalent RTL rewrites, since a human can typically only try one idea per iteration; and knowledge gained while fixing one design’s timing bottlenecks is usually **not systematically reused** on the next design, so similar problems are re-solved from scratch project after project.

4 Where AI Comes In

Generative AI – specifically, an **agentic LLM system grounded in real EDA tool feedback** – can directly target these bottlenecks. Rather than asking an LLM to blindly rewrite RTL from a natural-language description (which risks hallucinated, non-synthesizable, or functionally incorrect code), the adopted approach treats RTL optimization as a **closed-loop search problem**: the AI proposes a change, a real EDA toolchain evaluates it (synthesis + timing + formal equivalence checking), and the verified result feeds back into the next decision. This mirrors – and automates – exactly the manual loop described above, but with three key advantages that directly address the problems it causes:

- **Parallel exploration.** Instead of one fix per iteration, multiple candidate RTL rewrites are generated and evaluated in parallel per iteration, dramatically increasing the fraction of the design space explored.
- **Fine-grained, tool-grounded feedback.** A dedicated timing-analysis agent works directly from real post-synthesis timing reports (not a single coarse WNS number), localizing root causes to specific RTL structures before any fix is proposed – reducing wasted, ineffective rewrites.
- **Self-improving skill accumulation.** By comparing parallel candidates under matched conditions (*group-relative skill learning*), the system distills which pattern-strategy pairs (e.g., “deep decode logic → pre-computation”) actually work, and reuses this interpretable knowledge across future designs – directly solving the “expertise is not reused” problem of manual optimization.

Guardrails are essential: every accepted rewrite must pass **formal equivalence checking** against the original RTL before it is allowed to replace it, ensuring the AI’s aggressive search for better timing never comes at the cost of correctness.

5 Proposed Model: Adapting Dr. RTL’s Agentic Architecture

The system we plan to implement is based on **Dr. RTL** (Fang et al., 2026), whose multi-agent architecture is shown in Figure 2. Starting from an initial RTL design $\mathcal{D}_0$, the framework performs K optimization iterations. At each iteration t , the orchestrator coordinates three specialized agents in a loop:

- **Timing Analysis Agent** – consumes the current design’s real post-synthesis timing report, selects the top- k worst-slack paths, traces each back to its start/end registers and intermediate RTL logic, and diagnoses the likely structural root cause (e.g., high fan-out, excessive combinational depth).
- **RTL Optimization Agent** – generates N candidate RTL rewrites in parallel, guided by the timing diagnosis and by matching against the accumulated **skill library** of known effective pattern→strategy

pairs; if no known skill matches, the LLM proposes a new transformation from the timing feedback directly.

- **Evaluation Agent** – deterministically re-runs synthesis and formal (sequential) equivalence checking for every candidate, returning objective WNS/TNS/Area/pass-fail metrics with no LLM reasoning involved, so that every accepted decision is grounded in verified tool output rather than model self-assessment.

The orchestrator scores every equivalence-passing candidate on a weighted combination of WNS, TNS, and area, promotes the best candidate $\mathcal{D}_{t+1}$ to the next round, and – via **group-relative skill learning** – compares all N parallel candidates from the same iteration against each other to identify which transformation strategies reliably outperform their peers under matched conditions. Successful pattern-strategy pairs are added to (or reinforced in) the shared skill library with a confidence score, while ineffective or equivalence-violating transformations are explicitly flagged as strategies to avoid. This turns every optimization run into a source of reusable, human-readable design knowledge, not just a one-off fix.

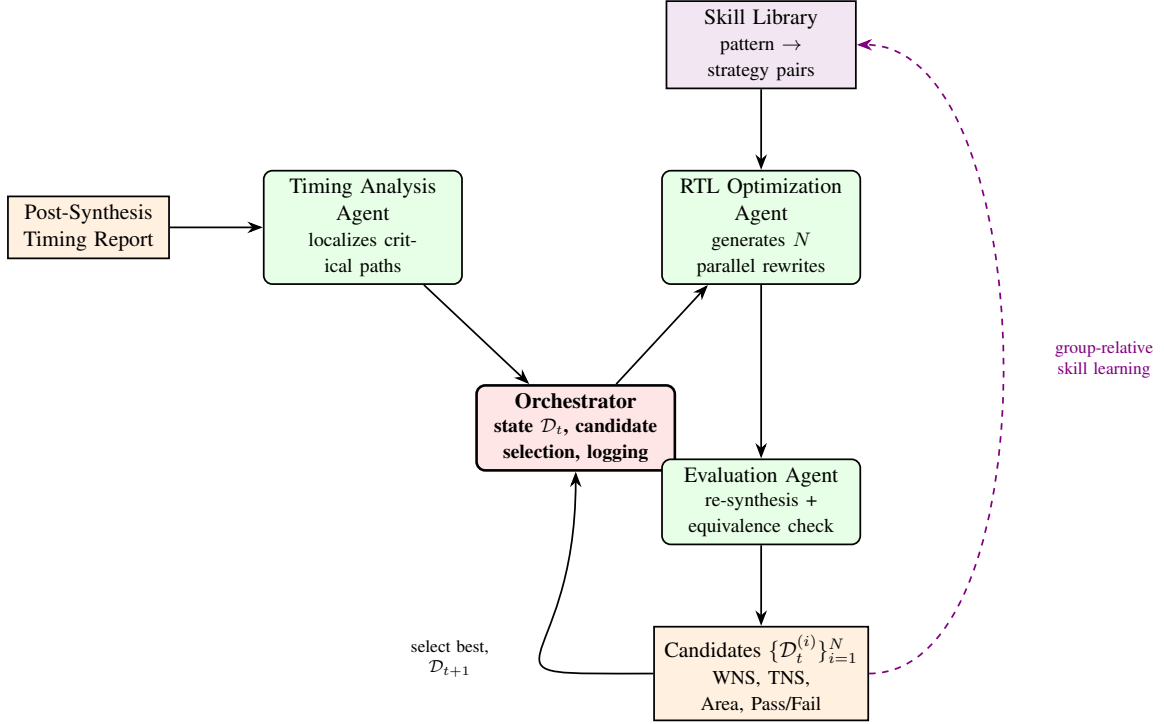


Figure 2: Agentic architecture (adapted from Dr. RTL): an orchestrator coordinates a Timing Analysis Agent, an RTL Optimization Agent, and a tool-grounded Evaluation Agent in a closed loop, while group-relative skill learning distills successful transformations from parallel candidates into a reusable skill library.

6 Expected Outcome

By adapting this methodology to an open-source EDA toolchain (Yosys for synthesis, OpenSTA for timing analysis, OpenROAD for physical design context, and EQY/SymbiYosys for formal equivalence checking), we aim to demonstrate a working, closed-loop GenAI system that measurably reduces timing violations (WNS/TNS) and area on realistic, multi-clock-domain RTL designs relative to synthesis alone – while guaranteeing functional correctness through formal verification at every step, and producing an interpretable, reusable log of what optimization strategies worked and why.